

Notebook Overview

- Notebook này sẽ hướng dẫn từng bước xây dựng một **pipeline hỏi đáp** (Q&A pipeline) trên một **thư mục chứa các tài liệu đầu vào**.
- 📁 **Dataset sử dụng:** [Google Drive Folder](https://drive.google.com/drive/folders/17RzKdSY6JbSd3JIHlueXGGpb_-sUyCSn?hl=vi) (https://drive.google.com/drive/folders/17RzKdSY6JbSd3JIHlueXGGpb_-sUyCSn?hl=vi).

```
In [ ]: from google.colab import drive  
drive.mount("/content/drive")
```

```
In [ ]: # Install required libraries  
!apt-get update  
!apt-get install -y libreoffice  
!pip install mammoth python-docx  
!apt-get install -y poppler-utils  
!pip install mammoth python-docx qdrant-client[fastembed] surya-ocr PyMuPDF pdf2image  
!pip install -U FlagEmbedding  
!pip install --upgrade together  
!pip install langchain
```

1.Process files in folder

1.1 Process file doc, docx

Hàm này dùng để **trích xuất nội dung văn bản từ file Word (.doc hoặc .docx)**.

Chức năng chính:

-  Nếu là `.doc` , sẽ tự động **chuyển sang `.docx`** bằng **LibreOffice** (chạy headless).
-  Sau đó, **trích xuất nội dung văn bản bằng Mammoth**
-  Nếu Mammoth thất bại, sẽ **fallback sang `python-docx`** để đọc nội dung.

```
In [ ]: import os, subprocess, logging
import mammoth
from docx import Document
import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
def parse_doc_file(file_path):
    def convert_doc_to_docx(path):
        out_dir = os.path.dirname(path)
        try:
            subprocess.run(["libreoffice", "--headless", "--convert-to", "docx", path, "--outdir", out_dir], check=True)
            new_path = path + ".docx"
            if os.path.exists(new_path):
                return new_path
            alt_path = os.path.join(out_dir, os.path.basename(path).replace(".doc", ".docx"))
            return alt_path if os.path.exists(alt_path) else None
        except Exception as e:
            logging.error(f"Convert failed: {e}")
            return None

    def extract_docx_text(path):
        try:
            doc = Document(path)
            return "\n".join(p.text for p in doc.paragraphs)
        except Exception as e:
            logging.error(f"Read .docx failed: {e}")
            return ''

    if file_path.endswith(".doc"):
        file_path = convert_doc_to_docx(file_path)
        if not file_path:
            return ''

    try:
        with open(file_path, "rb") as f:
            result = mammoth.extract_raw_text(f)
            return [result.value.strip()]
    except Exception:
        logging.warning("Mammoth failed, fallback to python-docx...")
        return extract_docx_text(file_path)
```

```
In [ ]: # Test with docx file
path = "/content/gdrive/MyDrive/AI_demo/datasets/example_docx.docx"
res = parse_doc_file(path)
```

```
In [ ]: res
```

1.2 Process file image (png, jpg,...)

 Trích xuất văn bản từ hình ảnh bằng Surya

Surya – một thư viện mã nguồn mở hỗ trợ cả:

- Text Detection
- Text Recognition
- Layout Analysis
- Table Detection
- LaTeX Extraction

Surya hoạt động hiệu quả trên nhiều loại tài liệu hình ảnh khác nhau và hỗ trợ đa ngôn ngữ.

 Tìm hiểu thêm tại: <https://github.com/VikParuchuri/surya> (<https://github.com/VikParuchuri/surya>)

```
In [ ]: from PIL import Image
from surya.recognition import RecognitionPredictor
from surya.detection import DetectionPredictor
import torch
import io
import os
```

```
In [ ]: # download model
recognition_predictor = RecognitionPredictor()
detection_predictor = DetectionPredictor()
langs = ["vi", "en"]
```

```
In [ ]: # Inference with image
def get_full_text(recognition_predictor, detection_predictor, image_bytes, langs):
    if isinstance(image_bytes, io.BytesIO):
        image_bytes = image_bytes.getvalue()

    image = Image.open(io.BytesIO(image_bytes))
    predictions = recognition_predictor([image], [langs], detection_predictor)
    full_text = "\n".join(line.text for line in predictions[0].text_lines)
    torch.cuda.empty_cache()

    return full_text

In [ ]: def parse_img_file(file_path, recognition_predictor, detection_predictor, langs):
    def process_image_file(path):
        if isinstance(path, str) and os.path.exists(path):
            with open(path, "rb") as f:
                return f.read()

        return

    image_byte = process_image_file(file_path)
    full_text = [get_full_text(recognition_predictor, detection_predictor, image_byte, langs)]
    return full_text

In [ ]: # test img file
path = "/content/gdrive/MyDrive/AI_demo/datasets/quy_che.png"
res = parse_img_file(path, recognition_predictor, detection_predictor, langs)
res
```

1.3 Process file PDF

Khi xử lý file PDF, cần phân biệt hai trường hợp phổ biến:

- ♦ **PDF không phải bản scan:** Sử dụng thư viện `fitz` / `PyMuPDF` để trích xuất trực tiếp nội dung văn bản được nhúng trong file.
- ♦ **PDF scan:** Tiến hành chuyển từng trang PDF thành ảnh, sau đó áp dụng **OCR** để trích xuất văn bản, theo quy trình đã mô tả ở bước trước.

```
In [ ]: import fitz # PyMuPDF
        from pdf2image import convert_from_bytes
```

```
In [ ]: # dùng fitz để parse thẳng nội dung từ file pdf
        def get_text_per_page(pdf_bytes):
            text_list = []
            doc = fitz.open(stream=pdf_bytes, filetype="pdf")
            for page in doc:
                text = page.get_text().strip()
                text_list.append(text)
            return text_list
```

```
In [ ]: def parse_pdf_file(file_path: str):
        full_text = []
        with open(file_path, "rb") as f:
            pdf_bytes = f.read()

        text_per_page = get_text_per_page(pdf_bytes)
        images = convert_from_bytes(pdf_bytes, dpi=300, fmt="png", thread_count=8)
        for idx, text in enumerate(text_per_page):
            if len(text) < 100:
                # Dùng 1 rule là nếu len text 1 trang < 100 => Scan => dùng OCR
                image_bytes = io.BytesIO()
                images[idx].save(image_bytes, format="PNG")
                image_bytes.seek(0)
                image_data = image_bytes.read()
                ocr_text = get_full_text(recognition_predictor, detection_predictor, image_data, langs)
                full_text.append(ocr_text)
            else:
                # Text PDF => dùng trực tiếp
                full_text.append(text.strip())

        return full_text
```

```
In [ ]: # example
        path = "/content/gdrive/MyDrive/AI_demo/datasets/pdf_non_scan.pdf"
        res = parse_pdf_file(path)
        res
```

1.4 Chunking

Để xử lý hiệu quả các tài liệu dài, chúng ta cần chia văn bản thành các đoạn nhỏ hơn — còn gọi là **chunk**. Đoạn code dưới sử dụng `RecursiveCharacterTextSplitter` của LangChain để:

- ✓ Chia văn bản đầu vào thành các đoạn có độ dài tối đa `chunk_size`.
- ↺ Mỗi đoạn sẽ **overlap** một số ký tự (`chunk_overlap`) để tránh mất ngữ cảnh.
- ✏ Tách đoạn ưu tiên tại các dấu kết thúc câu như `"."`, `"!"`, `"?"` để tránh chia giữa chừng câu.

Việc chunking này là bước quan trọng để đảm bảo chất lượng embedding và độ chính xác khi thực hiện truy vấn văn bản (Q&A hoặc tìm kiếm ngữ nghĩa).

Xem chi tiết hơn tại : https://python.langchain.com/docs/how_to/recursive_text_splitter/ (https://python.langchain.com/docs/how_to/recursive_text_splitter/).

```
In [ ]: from langchain.text_splitter import RecursiveCharacterTextSplitter

def chunk_text_with_overlap(text: str, chunk_size: int = 500, overlap: int = 50) -> list:
    """
    Chunk the input text into smaller segments, avoiding sentence breaks and including overlapping parts.

    Args:
        text (str): The input text to be chunked.
        chunk_size (int): Maximum length of each chunk.
        overlap (int): Number of characters to overlap between chunks.

    Returns:
        list: A list of chunked text segments.
    """
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=overlap,
        separators=[".", "!", "?"]
    )
    chunks = splitter.split_text(text)
    return chunks
```

```
In [ ]: # Example
text = """Hôm nay trời đẹp quá! Tôi quyết định đi dạo phố. Trên đường đi, tôi gặp một người bạn cũ.
Chúng tôi đã trò chuyện rất vui vẻ. Sau đó, tôi ghé vào một quán cà phê và thưởng thức một ly cà phê sữa đá.
Cuối cùng, tôi về nhà với tâm trạng rất thoải mái.
Hôm sau, tôi lại tiếp tục công việc của mình với năng lượng tràn đầy."""

# Chunk the text
chunks = chunk_text_with_overlap(text, chunk_size=100, overlap=20)

# Print the chunks
for i, chunk in enumerate(chunks):
    print(f"Chunk {i + 1}: {chunk}")
```

2.Embedding and save to Vector Embedding

⚙️ **Model embedding: BGE-M3 (<https://huggingface.co/BAAI/bge-m3>)**

- Sử dụng BGEM3FlagModel từ thư viện FlagEmbedding để embed văn bản thành vector.

🧠 **Vector Database: Qdrant (In-Memory)**

- Dùng QdrantClient(":memory:") để khởi tạo một vectorDB tạm thời trong RAM

🧩 **Chức năng chính của class:**

- encode(text) : Trả về vector sau khi được embedding.
- insert(texts) : Thêm các vector này vào Qdrant
- search(query_text, limit) : Truy vấn Qdrant để tìm các văn bản có độ tương đồng cao nhất.

```

In [ ]: from qdrant_client import QdrantClient, models
        from FlagEmbedding import BGEM3FlagModel
        from qdrant_client.models import PointStruct
        import numpy as np
        import uuid
        # Define Class QdrantSearch_bge
        class QdrantSearch_bge:
            def __init__(self, collection_name: str, model_name: str = "BAAI/bge-m3", use_fp16: bool = True):
                self.client = QdrantClient(":memory:") # Using Qdrant in mode in-memory
                self.collection_name = collection_name
                self.model = BGEM3FlagModel(model_name, use_fp16=use_fp16)

                # Create collection if not exist
                if not self.client.collection_exists(self.collection_name):
                    self.client.create_collection(
                        collection_name=self.collection_name,
                        vectors_config=models.VectorParams(size=1024, distance=models.Distance.COSINE)
                    )

            # embedding document
            def encode(self, text: str):
                emb = self.model.encode(text, return_dense=True, return_sparse=False, return_colbert_vecs=False)
                return emb['dense_vecs']

            # add embedding vectors into qdrant
            def insert(self, texts: list):
                points = []
                for idx, text in enumerate(texts):
                    dense_vec = self.encode(text)
                    points.append(
                        PointStruct(
                            id=uuid.uuid4().hex,
                            vector=dense_vec,
                            payload={"text": text}
                        )
                    )
                self.client.upsert(collection_name=self.collection_name, points=points)

            # function search
            def search(self, query_text: str, limit: int = 10):
                dense_vec = self.encode(query_text)
                results = self.client.search(

```

```
        collection_name=self.collection_name,  
        query_vector=dense_vec,  
        limit=limit,  
        with_payload=True  
    )  
    return results
```

```
In [ ]: # Process trên toàn bộ folder đầu vào
import tqdm

def process_folder(
    folder_path: str,
    searcher: QdrantSearch_bge,
    chunk_size: int = 800,
    overlap: int = 50,
):
    """
    Duyệt toàn bộ file trong folder, trích xuất văn bản, chunk và insert vào Qdrant.

    Args:
        folder_path (str): Đường dẫn đến thư mục chứa file.
        searcher (QdrantSearch_bge): Đối tượng Qdrant để insert chunk.
        chunk_size (int): Độ dài tối đa mỗi chunk.
        overlap (int): Ký tự chồng lặp giữa các chunk.
    """
    supported_images = [".png", ".jpg", ".jpeg"]
    supported_docs = [".doc", ".docx"]
    supported_pdfs = [".pdf"]

    for filename in tqdm.tqdm(os.listdir(folder_path)):
        print(f"Processing file: {filename}")
        file_path = os.path.join(folder_path, filename)
        ext = os.path.splitext(filename)[1].lower()

        extracted_texts = []

        if ext in supported_pdfs:
            extracted_texts = parse_pdf_file(file_path)
        elif ext in supported_docs:
            extracted_texts = parse_doc_file(file_path)
        elif ext in supported_images:
            extracted_texts = parse_img_file(file_path, recognition_predictor, detection_predictor, langs)
        else:
            print(f"🚫 Skipped unsupported file: {filename}")
            continue

        # Chunk và insert into Qdrant
        for text in extracted_texts:
```

```
chunks = chunk_text_with_overlap(text, chunk_size=chunk_size, overlap=overlap)
searcher.insert(chunks)

print(f"Processing and indexing sucessful")
```

```
In [ ]: folder_path = "/content/gdrive/MyDrive/AI_demo/dataset_2"
searcher = QdrantSearch_bge(collection_name="demo_2")
process_folder(folder_path, searcher)
```

```
In [ ]: # check số lượng bản ghi trong 1 collection trong qdrant
count = searcher.client.count(
    collection_name=searcher.collection_name,
    exact=True
).count

print(f"📦 Số lượng bản ghi hiện tại trong collection '{searcher.collection_name}': {count}")
```

3.Retrieval and generate answer

Hàm `pipeline()` thực hiện một flow RAG cơ bản gồm hai bước:

1 Retrieval

- Sử dụng `QdrantSearch_bge` để **truy vấn các đoạn văn bản liên quan** đến câu hỏi từ vector database Qdrant.
- Ghép các đoạn ngữ cảnh lại để cung cấp đầu vào cho mô hình ngôn ngữ.

2 Generation

- Gọi API từ **Together.ai** để sinh ra câu trả lời cuối cùng dựa trên truy vấn và ngữ cảnh đã truy xuất.
- Sử dụng: `Qwen/Qwen2.5-7B-Instruct-Turbo`.

Together.ai

- Là nền tảng cung cấp truy cập đến nhiều mô hình LLM mã nguồn mở
- Đăng ký tài khoản tại: <https://www.together.ai/> (<https://www.together.ai/>).
- 👉 Sau khi thêm credit card => sẽ nhận được **\$1 credit free**.

```
In [ ]: from together import Together
        from typing import List

        def pipeline(
            query: str,
            searcher: QdrantSearch_bge,
            top_k: int = 5,
            model: str = "Qwen/Qwen2.5-7B-Instruct-Turbo"
        ) -> str:
            # Step 1: Retrieval context from Qdrant
            results = searcher.search(query_text=query, limit=top_k)
            context_chunks = [hit.payload['text'] for hit in results if 'text' in hit.payload]
            context = "\n---\n".join(context_chunks)

            # Step 2: Call Together AI for generating answer
            client = Together()
            messages = [
                {"role": "user", "content": f"Thông tin liên quan:\n{context}\n\nCâu hỏi: {query}"}
            ]
            response = client.chat.completions.create(
                model=model,
                messages=messages
            )
            return response.choices[0].message.content
```

```
In [ ]: query = "Yêu cầu kinh kinh nghiệm của job được mô tả thế nào ?"

        ans = pipeline(query, searcher)
        ans
```

```
In [ ]: query = "Ứng viên Xuân Phúc có kinh nghiệm làm việc thế nào ?"

        ans = pipeline(query, searcher)
        ans
```

```
In [ ]: !jupyter nbconvert --clear-output --inplace /content/drive/MyDrive/AI_demo/demo.ipynb  
!jupyter nbconvert --to html --template classic /content/drive/MyDrive/AI_demo/demo.ipynb
```

```
In [ ]: from google.colab import files  
files.download('/content/drive/MyDrive/AI_demo/demo.html')
```

```
In [ ]:
```